

AIR QUALITY ANALYTICS

2602096823 - RAZKA NADHIF PUTRA PRADEP

2802418916 - JERSEY LEOTA WISESA

2802417106 - KENNARD YULIUS

2802407263 - VINSON MERVIN

2802413291 - JOSHUA FERDINAND

2802407414 - SYLVESTER LUIGI HIDAYAT

2802410346 - JULIUS GILANG NUGROHO

2802411866 - HANSEL LIE ARIFIN

BACKGROUND

This dataset is sourced from Kaggle with topic, "Air Quality Data in Brooklyn, NY – OpenAQ 2025". This data contains **air quality records**, specifically severe air quality levels on the PM2.5 parameter, timestamps, and locations from various monitoring stations in the Brooklyn, New York area. This dataset consist of 2 csv files.

Dataset Source: <https://www.kaggle.com/datasets/aidenl25/air-quality-data-in-brooklyn-ny-openaq-2025>

BUSINESS DOMAIN & PROBLEM STATEMENT

- Air pollution is influenced by:
 - Traffic
 - Industrial activities
 - Weather conditions
- Traditional systems often struggle to process large amounts of realtime data from IoT air quality sensors.
- A scalable Big Data system is needed to:
 - Collect data efficiently
 - Store continuous pollution data
 - Analyze pollution data
- This system can help:
 - Detect pollution spikes
 - Identify high-risk areas
 - Support faster environmental monitoring and public health decision-making

BIG DATA CHARACTERISTICS

(5V ANALYSIS)

Dimension	Aplication to Air Quality Data
Volume	The dataset contains 1,392 hourly PM2.5 records from Brooklyn, representing continuously generated air quality data that may scale over time.
Velocity	Air quality sensors continuously generate hourly PM2.5 data, requiring realtime ingestion and processing infrastructure.
Variety	The dataset includes numerical, timestamp, geographic, and categorical data related to air quality monitoring.
Veracity	The dataset contains missing values and data quality inconsistencies, highlighting the importance of preprocessing and data cleaning before analysis.
Value	The processed data can support environmental policies, pollution monitoring, and public health awareness.

BUSINESS MOTIVATIONS & EXPECTED VALUE

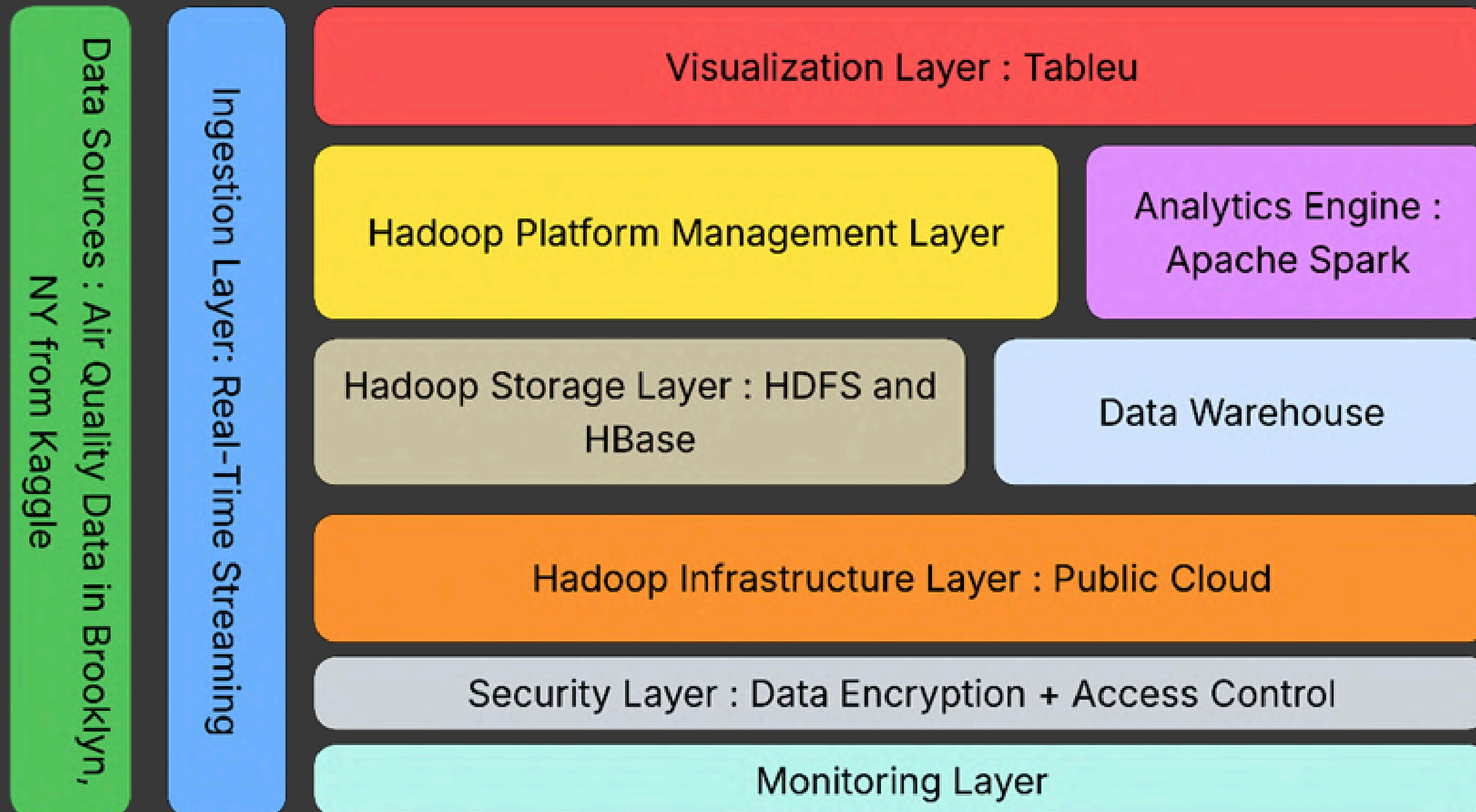
Business Motivations : Increasing urban air pollution creates challenges for environmental monitoring and public health management. Implementing a Big Data architecture enables faster analysis and supports data-driven decision-making to improve environmental awareness and urban sustainability.

Expected Value:

- Faster and more accurate decision-making
- Pollution forecasting
- Better resource allocation in high-risk areas
- Support for sustainable smart city development

BIG DATA ARCHITECTURE OVERVIEW

Big Data Architecture



DATA SOURCES

- 
- **Measurements.csv** contains **1,391 hourly PM2.5** readings collected from two monitoring stations between August 1–30, 2025. Each record includes **sensor details, timestamps, location coordinates**, and **PM2.5 values in $\mu\text{g}/\text{m}^3$** . Several data quality **issues were found**, such as the **PM2.5 column being stored as a string**, **missing values in isMobile, isMonitor**, and **country_iso**, and a **duplicate header row** caused by data merging.
 - **Stations.csv** provides **metadata** for the **two monitoring stations**, including **station ID, station name, coordinates, sensor type**, and **data source**. Station 1 (Near Bay 50 St) uses an AirGradient sensor, while Station 2 (Bklyn - PS 314) uses a reference-grade AirNow monitor.

INGESTION LAYER

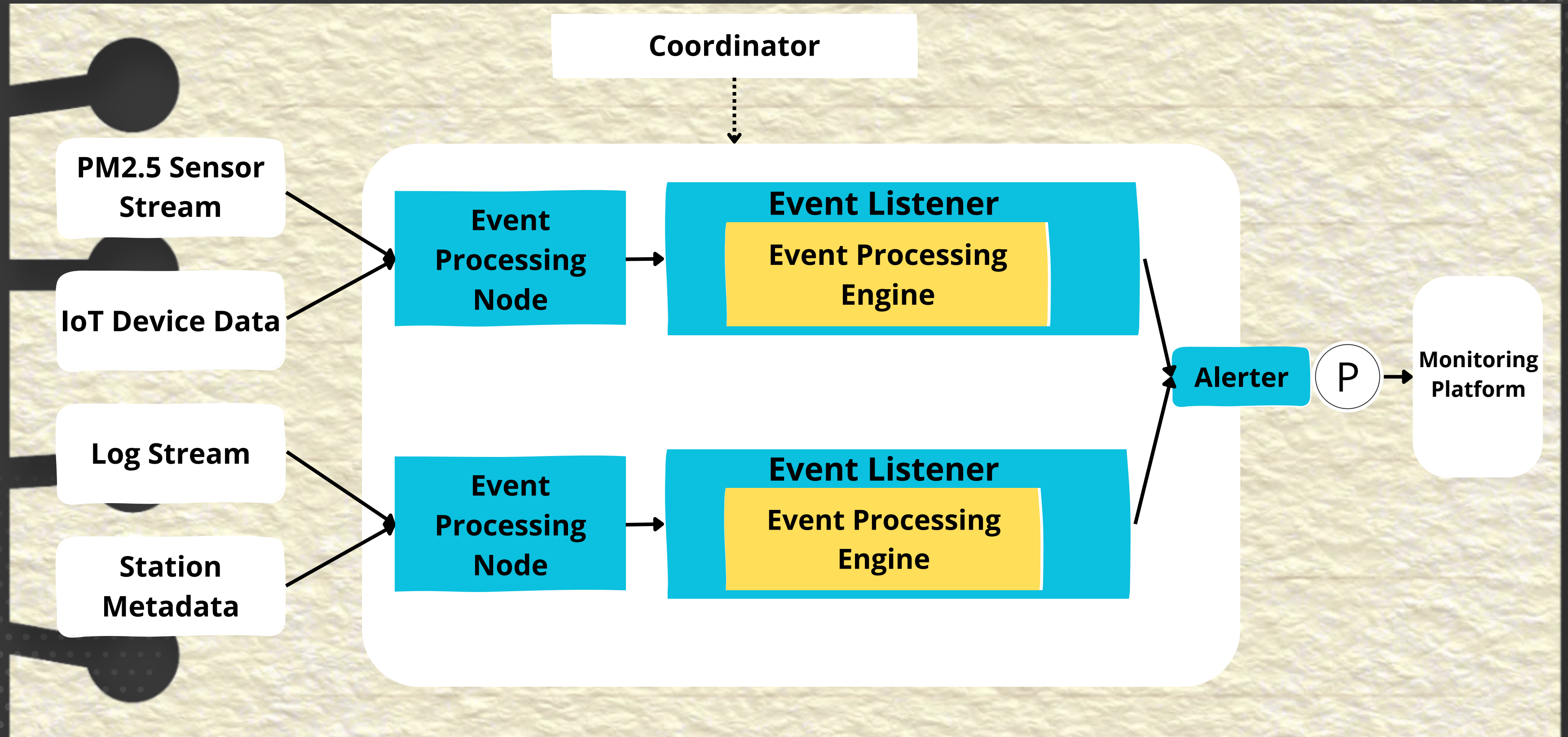


The system adopts streaming ingestion to continuously collect and process realtime PM2.5 data from IoT sensors, enabling timely monitoring and faster decision-making.

By adopting a real time streaming ingestion pattern, the system is able to:

- Process sensor data in real time
- Detect pollution spikes instantly
- Support live monitoring dashboards
- Generate PM2.5 threshold alerts
- Enable fast analytics and decision making

REAL TIME STREAMING PATTERN



STORAGE LAYER

HDFS

- Primary Distributed Storage: Acts as the main archive for massive, long-term historical PM 2.5 sensor datasets.
- Batch Analytics: Optimized for large-scale batch processing to analyze long-term air quality trends.
- Horizontal Scalability: Easily expands by adding virtual cloud nodes as continuous sensor data grows over time.
- Fault-Tolerant: Automatically replicates data (typically 3 times) across multiple nodes to prevent permanent data loss if a server fails.

STORAGE LAYER

HBase

- NoSQL Database: Built on top of HDFS, specifically designed for high-velocity, time-stamped IoT sensor data.
- Real-Time Operations: Enables ultra-fast read and write operations with very low latency.
- Continuous Ingestion: Efficiently handles continuous, high-speed PM 2.5 streaming data from monitoring stations.
- Live Actionable Insights: Directly powers live dashboards and automated pollution alerts for quick responses.

STORAGE LAYER

Why HDFS + HBase?

- HDFS provides scalable and fault-tolerant storage for large historical datasets.
- HBase enables low-latency access for realtime sensor data.
- Both technologies support horizontal scalability as the number of sensors grows.
- Together, they efficiently process continuous IoT streaming data from the ingestion pipeline.
- This architecture supports both historical data storage and realtime analytics, making it suitable for smart city and environmental monitoring systems.

PROCESSING & ANALYTICS ENGINE

Why Apache Spark?

1

Supports both batch and realtime analytics in one framework.

2

Provides fast processing through in-memory computation.

3

Seamlessly integrates with Hadoop, supporting HDFS and HBase data access.

4

Offers Spark SQL and Structured Streaming APIs for flexible real-time and batch processing.

PROCESSING & ANALYTICS ENGINE

Processing Flow

1

PM2.5 sensor data is collected continuously from IoT monitoring stations.

2

Apache Spark Structured Streaming processes realtime and historical data from HDFS and HBase.

3

The processed data is used for realtime dashboards, pollution monitoring, and environmental analysis.

VISUALIZATION LAYER



The visualization layer uses Tableau to present interactive and user-friendly PM2.5 data insights, including pollution trends and time-series patterns. Tableau connects directly to the processed data from Apache Spark, ensuring the dashboards display clean, structured, and analysis-ready information.

MONITORING & SECURITY



SECURITY LAYER

The security layer protects air quality sensor data across ingestion, storage, and analytics components by securing data transmission and storage to prevent unauthorized access, data tampering, and other security threats.

- **Data Encryption**

Data encryption is used to protect PM2.5 sensor data both when the data is stored and when it is transferred between systems. HDFS uses Hadoop Transparent Data Encryption (TDE) to secure stored data, while SSL/TLS protects data transmission and helps prevent unauthorized access or data interception.

MONITORING & SECURITY

- **Access Control**

Access control is used to **manage who can view and use the data** in the **system**. Different users are given different access rights based on their roles. For example, the system can process and manage data, while users can only see the final results through approved dashboards. Overall, the security layer helps keep air quality data safe throughout the entire process

Monitoring Layer

The monitoring layer **monitors Hadoop, HBase, and Spark performance** using tools such as Apache Ambari, Prometheus, and ELK Stack for realtime monitoring, alerting, and log analysis.

MONITORING & SECURITY



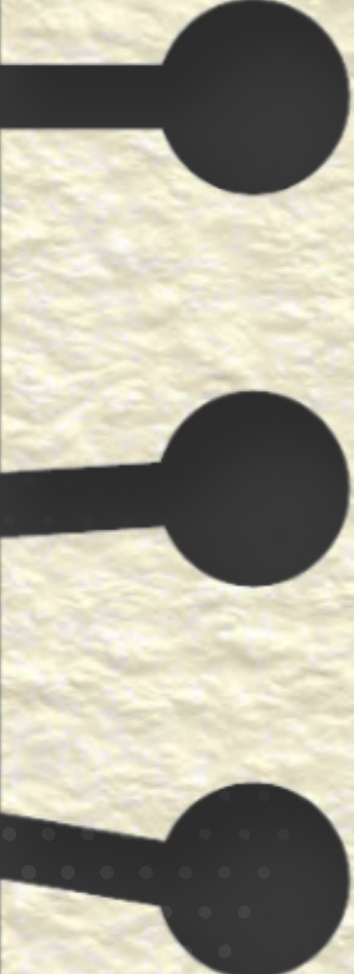
Centralized Log Collection

The monitoring layer keeps the Big Data system reliable and responsive through continuous monitoring, realtime alerts, and centralized log management.

Cluster and Infrastructure Monitoring

The system **monitors overall performance**, including **resource usage and system activity**, to ensure **stable operation**. Monitoring tools help **detect issues early** and **provide visibility** into system performance through dashboards.

DEPLOYMENT MODEL & ARCHITECTURE USED



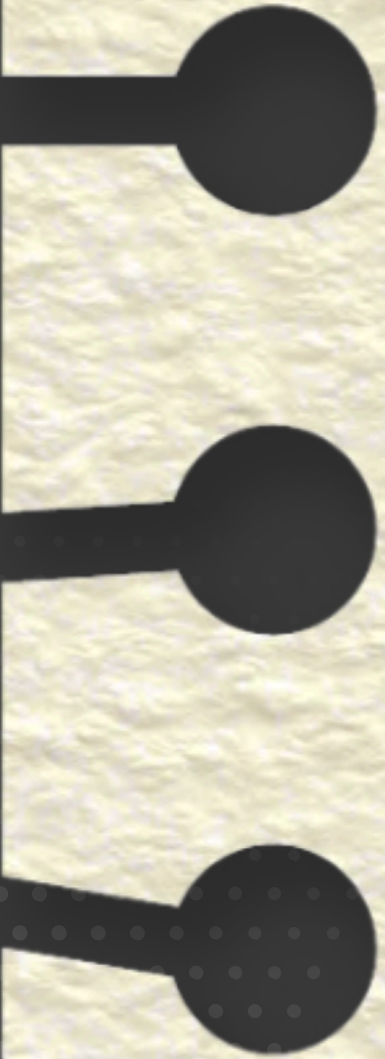
Deployment Model

The architecture uses a public cloud model on Google Cloud Platform because it provides scalable resources, lower infrastructure costs, and easier system management without physical hardware.

Virtual Infrastructure vs Physical Data Center

Cloud-based virtual infrastructure is chosen because it offers flexible scalability, faster deployment, and better cost efficiency for processing growing PM2.5 sensor data compared to traditional physical data centers.

IMPLEMENTATION DETAILS – BIG DATA ANALYTICS



The analytics process uses **Apache Spark (PySpark)** on Google Colab to process Brooklyn air quality data from measurements.csv and stations.csv. The pipeline includes data preprocessing, descriptive analytics, and classification modeling.

Descriptive Analytics

- **Pollution Level Distribution:** Counts PM2.5 categories (Good, Moderate, Unhealthy) and visualizes them using a pie chart.
- **Average PM2.5 by Provider:** Calculates average PM2.5 values by provider and displays them in a bar chart.
- **Daily Average PM2.5:** Analyzes daily PM2.5 trends using a bar chart, showing pollution peaks around day 5 and day 6 before gradually decreasing.

CLASSIFICATION MODEL – RANDOM FOREST CLASSIFIER

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

```
accuracy_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="accuracy"  
)
```

```
accuracy = accuracy_evaluator.evaluate(predictions)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9166666666666666
```

```
f1_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="f1"  
)
```

```
f1 = f1_evaluator.evaluate(predictions)
```

```
print("F1 Score:", f1)
```

```
F1 Score: 0.9151069964508189
```

A Random Forest Classifier

using PySpark MLlib is trained to predict pollution levels (Good, Moderate, Unhealthy) based on hour, month, latitude, and longitude, day features. The dataset is split into **80% training** and **20% testing** data, while model performance is evaluated using Accuracy and F1 Score.

CLASSIFICATION MODEL – RANDOM FOREST CLASSIFIER

Random Forest Classifier was used because it can capture spatial and temporal patterns in air quality data while maintaining high classification accuracy.

Feature Cols Used:

- latitude
- longitude
- month
- day
- hour

```
from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(
    labelCol="label",
    featuresCol="features",
    numTrees=100,
    maxDepth=10,
    seed=42
)

model = rf.fit(train_data)
```

CLASSIFICATION MODEL – RANDOM FOREST CLASSIFIER

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

```
accuracy_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="accuracy"  
)
```

```
accuracy = accuracy_evaluator.evaluate(predictions)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9166666666666666
```

```
f1_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="f1"  
)
```

```
f1 = f1_evaluator.evaluate(predictions)
```

```
print("F1 Score:", f1)
```

```
F1 Score: 0.9151069964508189
```

Accuracy

```
[ ] from pyspark.ml.evaluation import MulticlassClassificationEvaluator  
  
accuracy_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="accuracy"  
)
```

```
accuracy = accuracy_evaluator.evaluate(predictions)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.6416666666666667
```

F1 Score

```
[ ] f1_evaluator = MulticlassClassificationEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="f1"  
)
```

```
f1 = f1_evaluator.evaluate(predictions)
```

```
print("F1 Score:", f1)
```

```
F1 Score: 0.5016074450084603
```

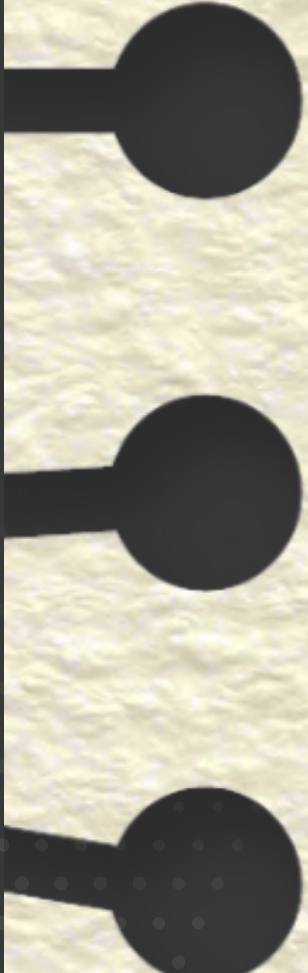
CLASSIFICATION MODEL – RANDOM FOREST CLASSIFIER

```
▶ for feature, importance in zip(  
    feature_cols,  
    model.featureImportances  
):  
    print(feature, ":", importance * 100, "%")
```

```
... latitude : 7.019850013788935 %  
longitude : 5.223586320134879 %  
month : 0.0 %  
day : 73.85749346630526 %  
hour : 13.899070199770918 %
```

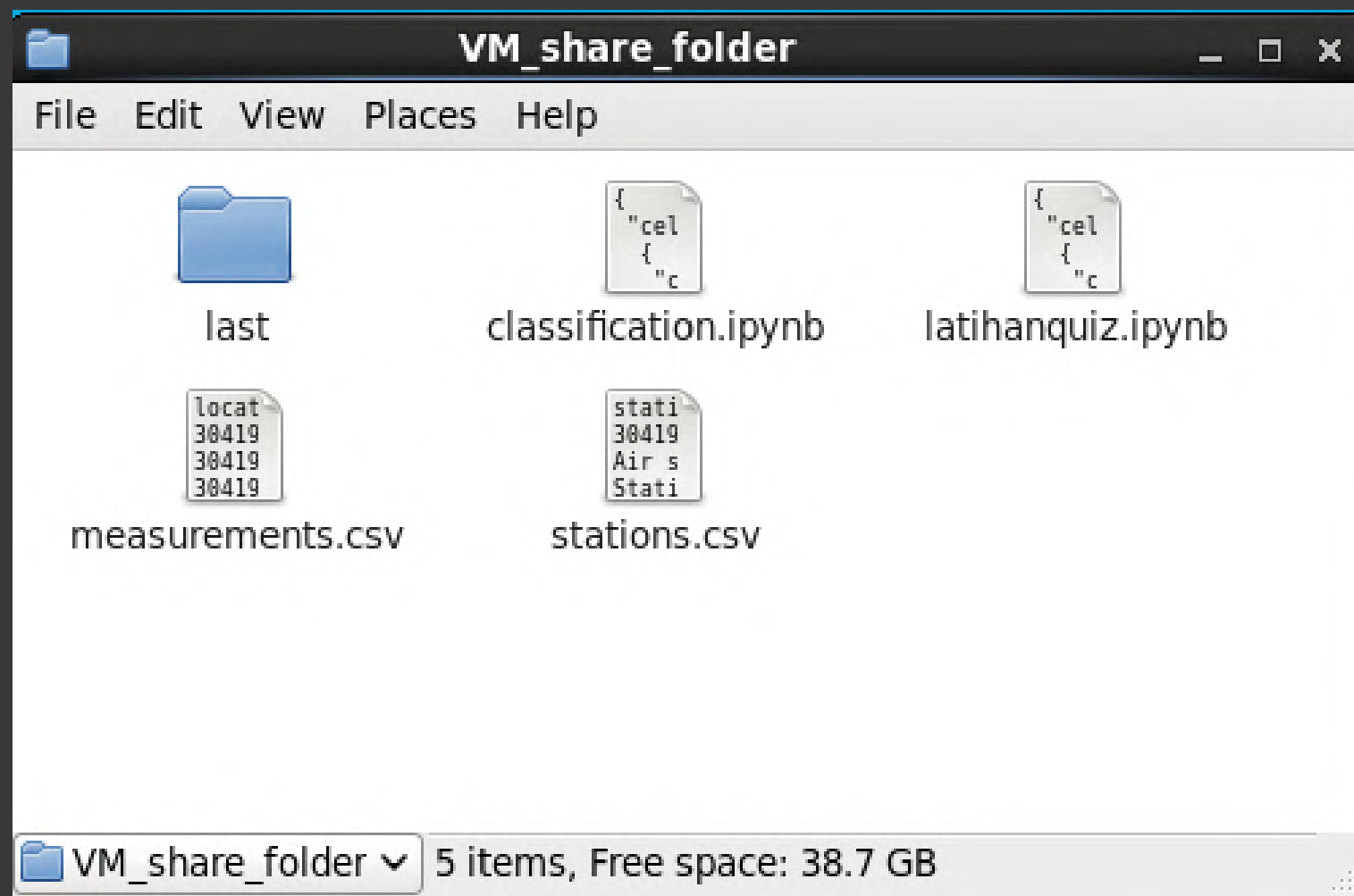
This code displays the feature importance values generated by the Random Forest model. The results show that day is the most influential feature, contributing 73.86% to the classification process.

INTERPRETATION OF RESULTS



Google Colab for Analytics using Spark:

https://colab.research.google.com/drive/13hZj9HrfV9rYGsM7x5wkF9u9_vny0zZ1?usp=sharing



```
cloudera@quickstart:~/Desktop
File Edit View Search Terminal Help
(base) [cloudera@quickstart Desktop]$ jupyter notebook
[I 10:33:06.290 NotebookApp] The port 8888 is already in use, trying another port.
[I 10:33:06.376 NotebookApp] JupyterLab extension loaded from /home/cloudera/Downloads/]lib/python3.7/site-packages/jupyterlab
[I 10:33:06.376 NotebookApp] JupyterLab application directory is /home/cloudera/Downloads/]share/jupyter/lab
[I 10:33:06.378 NotebookApp] Serving notebooks from local directory: /home/cloudera/Desktop
[I 10:33:06.378 NotebookApp] The Jupyter Notebook is running at:
[I 10:33:06.379 NotebookApp] http://localhost:8889/?token=568dd085b25808ba27a3ef56e0c8d9c4b829186567762bdb
[I 10:33:06.379 NotebookApp] or http://127.0.0.1:8889/?token=568dd085b25808ba27a3ef56e0c8d9c4b829186567762bdb
[I 10:33:06.379 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 10:33:06.459 NotebookApp]

To access the notebook, open this file in a browser:
    file:///home/cloudera/.local/share/jupyter/runtime/nbserver-6853-open.html
Or copy and paste one of these URLs:
    http://localhost:8889/?token=568dd085b25808ba27a3ef56e0c8d9c4b829186567762bdb
```

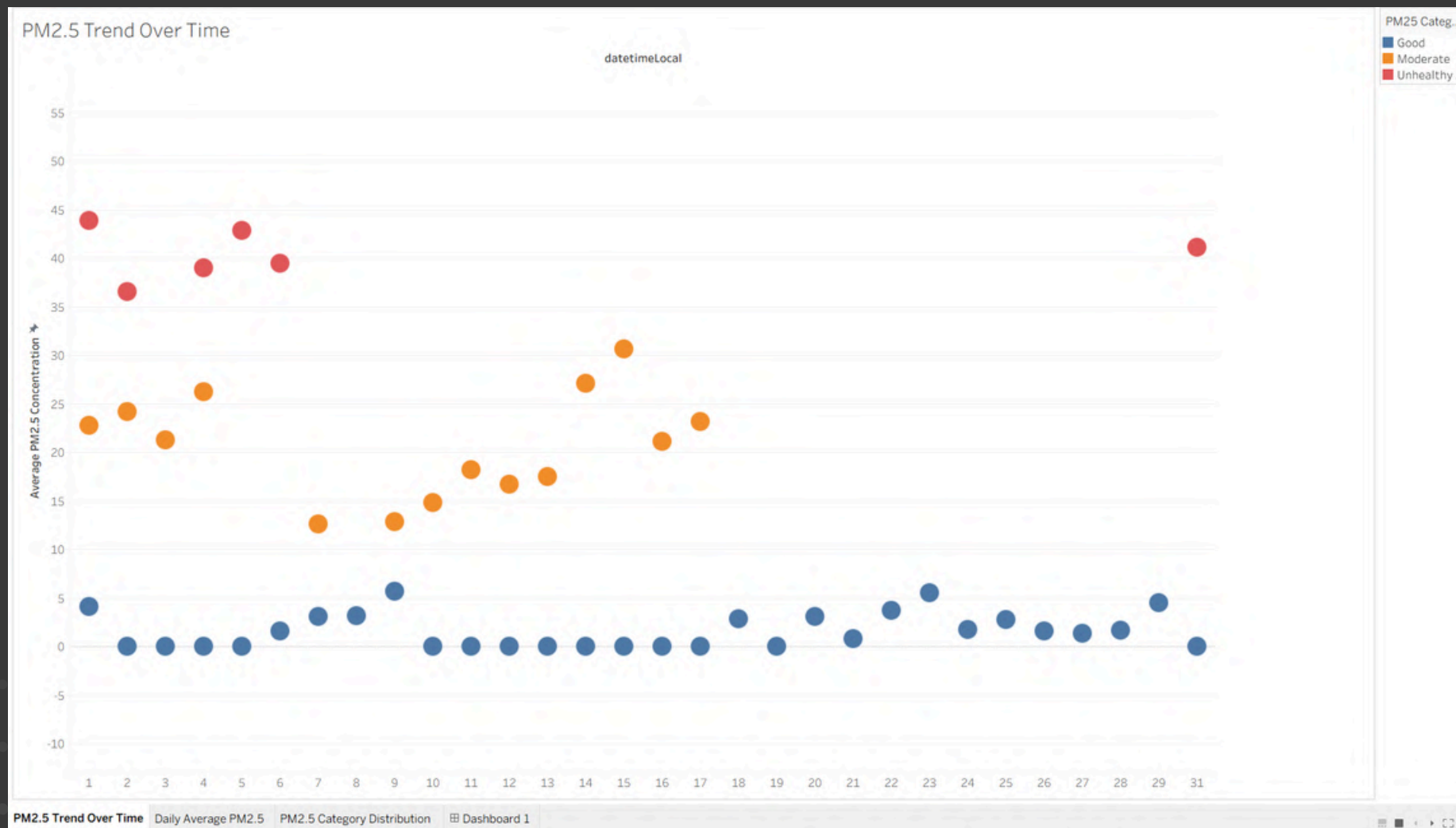
```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
```

```
measurements = spark.read.csv("measurements.csv", header=True, inferSchema=True)
stations = spark.read.csv("stations.csv", header=True, inferSchema=True)
```

```
measurements.registerTempTable("measurements")
stations.registerTempTable("stations")
```

VISUALIZATION 1:

PM2.5 Changes Over Time by Category



Key Insights:

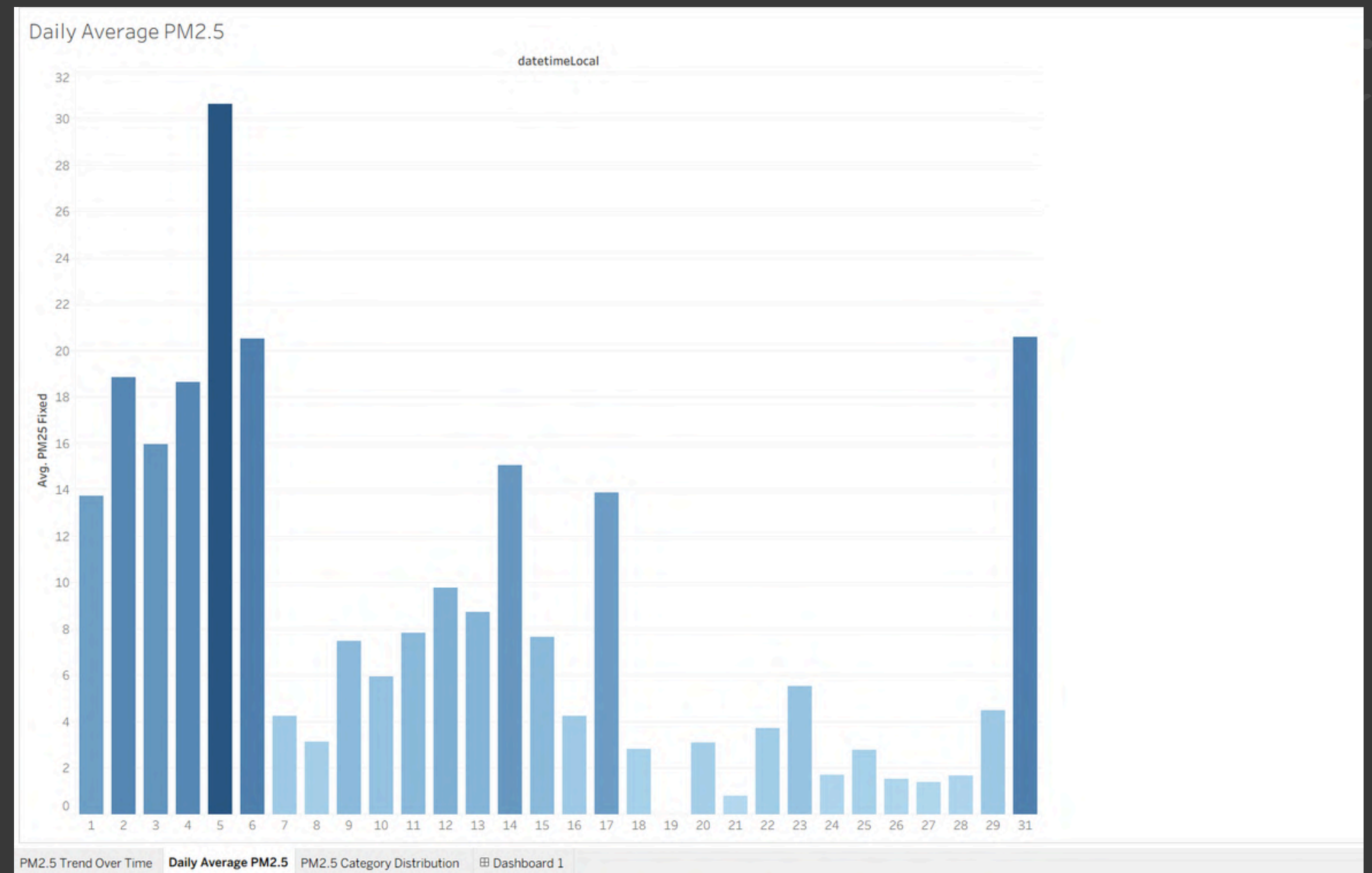
- PM2.5 levels fluctuate daily during the observation period.
- Pollution spikes indicate temporary increases in air pollution.
- Declining PM2.5 periods reflect improved air quality.
- Air quality changes are influenced by environmental and human activity factors.
- PM2.5 monitoring helps identify pollution trends and support analysis.

VISUALIZATION 2:

Daily Average PM2.5

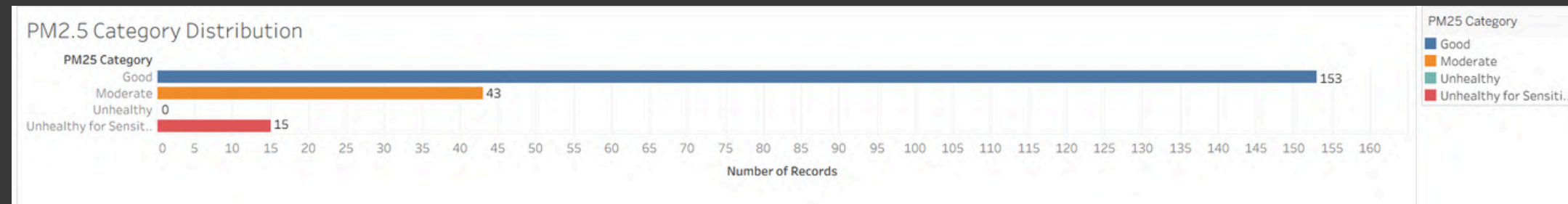
Key Insights:

- PM2.5 concentrations vary across different days.
- Moderate pollution levels occur most frequently.
- Some days reach unhealthy pollution levels for sensitive groups.
- Daily comparisons help identify periods of higher pollution exposure.
- The data supports environmental monitoring and public health awareness.



VISUALIZATION 3:

PM2.5 Category Distribution



Key Insights:

- Most PM2.5 measurements are categorized as "Good."
- Moderate pollution levels still occur regularly.
- Only a small portion reaches unhealthy pollution levels.
- Air quality generally remains safe but still requires monitoring and preventive action.

CONCLUSION FOR VISUALIZATION

- 
- PM2.5 concentrations fluctuated over time, showing dynamic air quality changes.
 - Some days experienced moderate to unhealthy pollution levels.
 - Most PM2.5 observations were still categorized as "Good."
 - Tableau helps visualize pollution trends and air quality distributions clearly.
 - Data visualization supports environmental monitoring and decision-making.

RESULT AND DISCUSSION

1. Result & Business Insights

The designed big data architecture successfully handles PM2.5 sensor data from ingestion to visualization using HDFS, HBase, Spark, and Tableau. The system supports efficient storage, fast processing, and interactive analysis of air quality data. From a business perspective, the architecture helps environmental agencies monitor pollution trends, detect PM2.5 spikes, and support faster public health decision-making in Brooklyn.

2. Challenges & Limitations

A major challenge is integrating multiple big data technologies into a stable and low-latency pipeline. Since the project uses a Kaggle dataset instead of live sensor feeds, the real-time ingestion process is only simulated. In addition, the architecture currently focuses only on monitoring and visualization because it does not yet include predictive analytics or machine learning features.

RESULT AND DISCUSSION

3. Future Improvements

Future improvements include integrating machine learning with Apache Spark to predict PM2.5 levels based on historical and weather data. The system could also be connected to live OpenAQ API feeds for real-time monitoring and expanded beyond Brooklyn to support wider air quality management across New York City.

THANK
YOU

